

■ ft_printf expliqué

Guide ultra-détaillé pour les 10 ans et plus ■

Ce document te explique **chaque fichier** de ton projet **ft_printf** (42 Piscine) comme si tu avais 10 ans. On va utiliser des analogies simples, des exemples concrets, et des schémas textuels pour que tu comprennes vraiment ce que fait chaque ligne de code.

#	Fichier	Rôle en une phrase
1	ft_printf.h	La liste de courses — déclare tout ce qui existe
2	ft_printf.c	Le chef d'orchestre — lit le format et distribue le travail
3	ft_print_char.c	Imprime un seul caractère
4	ft_print_str.c	Imprime une chaîne de caractères
5	ft_print_nbr.c	Imprime un nombre entier (positif ou négatif)
6	ft_print_unsigned.c	Imprime un nombre sans signe (toujours positif)
7	ft_print_hex.c	Imprime un nombre en hexadécimal (base 16)
8	ft_print_ptr.c	Imprime une adresse mémoire (pointeur)
9	ft_print_percent.c	Imprime juste le symbole %
10	Makefile	La recette de construction du projet

■ C'est quoi printf, et pourquoi le recoder ?

Tu connais déjà **printf** : c'est la fonction magique en C qui affiche des choses à l'écran. Par exemple :

```
printf("Bonjour %s, tu as %d ans !\n", "Zain", 20);
```

Ca affiche : **Bonjour Zain, tu as 20 ans !**

Le **%s** est remplacé par "Zain", et **%d** par 20. C'est ce qu'on appelle un *spécificateur de format*.

À la 42, on te demande de **recoder cette fonction toi-même** pour comprendre comment elle marche à l'intérieur. C'est comme si on te demandait de démonter une montre et de la remonter pièce par pièce. Ton projet s'appelle **ft_printf**.

■ Concept clé : les arguments variables (variadic functions)

printf peut recevoir 1, 2, 5 ou 100 arguments. Comment ? Grâce à **va_list**, **va_start**, **va_arg**, **va_end** — des outils C qui permettent de lire des arguments en nombre inconnu. On y revient en détail dans **ft_printf.c**.

1 — ft_printf.h · Le fichier d'en-tête (header)

■ Analogie : la liste de courses

Imagine que tu prépares un grand repas avec plein d'amis cuisiniers. Avant de commencer, tu affiches au mur une **liste** qui dit : "Il y aura une fonction pour les chars, une pour les strings, une pour les nombres..." Chaque cuisinier sait ainsi ce qui existe et ce qu'il peut utiliser. C'est exactement le rôle du fichier **.h** (header).

■ Le code complet

```
#ifndef FT_PRINTF_H
# define FT_PRINTF_H

# include <stdarg.h>
# include <unistd.h>
# include <stdlib.h>

int  ft_printf(const char *format, ...);
int  ft_print_char(int c);
int  ft_print_str(char *s);
int  ft_print_nbr(int n);
int  ft_print_unsigned(unsigned int n);
int  ft_print_hex(unsigned int n, const char format);
int  ft_print_ptr(unsigned long ptr);
int  ft_print_percent(void);

#endif
```

■ Explication ligne par ligne

■ #ifndef FT_PRINTF_H / #define FT_PRINTF_H / #endif

C'est une **garde d'inclusion** (include guard). Problème qu'elle résout : si deux fichiers .c incluent tous les deux ft_printf.h, sans cette garde, les déclarations seraient copiées deux fois, ce qui ferait une erreur. Avec la garde : la première fois, FT_PRINTF_H n'existe pas donc on continue. La deuxième fois, il existe déjà donc on saute tout le contenu. C'est comme mettre un panneau 'DÉJÀ LU' sur un document.

■ #include <stdarg.h>

Cette bibliothèque fournit les outils pour lire des arguments en nombre variable : va_list, va_start, va_arg, va_end. Sans elle, impossible de savoir combien d'arguments ont été passés à ft_printf.

■ #include <unistd.h>

Fournit la fonction **write()**, qui est le seul moyen d'écrire à l'écran autorisé par la 42 (pas de putchar ni de printf de la libc !).

■ #include <stdlib.h>

Fournit des utilitaires de base comme malloc/free. Ici il est inclus par précaution, même si le code actuel ne l'utilise pas directement.

■ Les lignes int ft_printf(...), int ft_print_char(...), etc.

Ce sont les **prototypes** (ou déclarations) de chaque fonction. Un prototype, c'est comme une promesse : 'Cette fonction existe quelque part, elle prend ces paramètres, et elle retourne un int.' Tous les fichiers .c qui veulent appeler une de ces fonctions doivent inclure ce header. Sans ça, le compilateur dirait 'je ne connais pas cette fonction !'

■ **Pourquoi les fonctions retournent int ?** Parce qu'elles renvoient le **nombre de caractères imprimés**. Si quelque chose tourne mal (erreur d'écriture), elles renvoient **-1**. Ainsi ft_printf peut lui aussi renvoyer le total.

2 — ft_printf.c · Le chef d'orchestre

■ Analogie : le distributeur de courrier

Imagine un employé de la poste. Il reçoit un grand sac de lettres. Il les lit une par une : si c'est une lettre normale, il la met dans la boîte aux lettres. Si c'est un colis (signalé par un '%'), il regarde ce qu'il y a après le '%' et l'envoie au bon département : département 's' pour les chaînes, département 'd' pour les nombres, etc. C'est exactement ce que fait ft_printf.

■ Le code complet

```
#include "ft_printf.h"

static int dispatch(char spec, va_list ap)
{
    if (spec == 'c')
        return (ft_print_char(va_arg(ap, int)));
    if (spec == 's')
        return (ft_print_str(va_arg(ap, char *)));
    if (spec == 'p')
        return (ft_print_ptr((unsigned long)va_arg(ap, void *)));
    if (spec == 'd' || spec == 'i')
        return (ft_print_nbr(va_arg(ap, int)));
    if (spec == 'u')
        return (ft_print_unsigned(va_arg(ap, unsigned int)));
    if (spec == 'x' || spec == 'X')
        return (ft_print_hex(va_arg(ap, unsigned int), spec));
    if (spec == '%')
        return (ft_print_percent());
    return (0);
}

static int put_raw(char c)
{
    if (write(1, &c, 1) == -1)
        return (-1);
    return (1);
}

int ft_printf(const char *format, ...)
{
    va_list ap;
    int i;
    int count;
    int sub;

    if (!format)
        return (-1);
    va_start(ap, format);
    i = 0;
```

```

    count = 0;
    while (format[i])
    {
        if (format[i] == '%' && format[i + 1])
            sub = dispatch(format[++i], ap);
        else
            sub = put_raw(format[i]);
        if (sub < 0)
            return (va_end(ap), -1);
        count += sub;
        i++;
    }
    va_end(ap);
    return (count);
}

```

■ Explication des trois fonctions

■ Fonction `ft_printf` (la principale)

C'est la fonction que l'utilisateur appelle. Elle prend un **format** (la chaîne avec les % dedans) suivi de ... (les trois petits points = nombre variable d'arguments).

■ `if (!format) return (-1);`

Vérification défensive : si on nous donne NULL au lieu d'une vraie chaîne, on retourne immédiatement -1. C'est comme vérifier qu'on a bien reçu une lettre avant de l'ouvrir.

■ `va_start(ap, format);`

On 'ouvre' la boîte d'arguments variables. `ap` (argument pointer) est comme un curseur qui pointe sur le premier argument après `format`. Sans cette ligne, on ne peut pas accéder aux arguments supplémentaires.

■ `while (format[i])`

On parcourt chaque caractère du format, un par un, jusqu'au caractère nul (fin de chaîne).

■ `if (format[i] == '%' && format[i + 1])`

Si on trouve un '%' ET qu'il y a un caractère après (on vérifie `format[i+1]` pour éviter de lire hors de la chaîne), c'est un spécificateur. On fait `++i` pour avancer sur la lettre après '%', puis on appelle `dispatch`.

■ `else sub = put_raw(format[i]);`

Sinon c'est un caractère normal (une lettre, un espace, un `\n...`), on l'écrit directement à l'écran avec `put_raw`.

■ `count += sub;`

On accumule le nombre total de caractères écrits. C'est ce total qu'on retourne à la fin.

■ **va_end(ap);**

'Ferme' la boîte d'arguments. Obligatoire pour libérer les ressources internes utilisées par va_start. Oublier va_end = comportement indéfini !

■ **Fonction dispatch (le tri postal)**

dispatch reçoit la lettre après le '%' (spec) et la liste d'arguments (ap). Elle regarde spec et appelle la bonne fonction :

Spécificateur	Fonction appelée	Exemple d'usage
%c	ft_print_char	ft_printf("%c", 'A') → A
%s	ft_print_str	ft_printf("%s", "bonjour") → bonjour
%p	ft_print_ptr	ft_printf("%p", ptr) → 0x7ffee1a2b3c4
%d ou %i	ft_print_nbr	ft_printf("%d", -42) → -42
%u	ft_print_unsigned	ft_printf("%u", 42) → 42
%x ou %X	ft_print_hex	ft_printf("%x", 255) → ff
%%	ft_print_percent	ft_printf("100%%") → 100%

Remarque : **va_arg(ap, int)** consomme le prochain argument de la liste et le lit comme un int. Chaque appel à va_arg avance le curseur. L'ordre est donc crucial : les arguments sont consommés dans l'ordre où les '%' apparaissent dans le format.

■ **Fonction put_raw (l'imprimante basique)**

Un simple wrapper autour de write(). Écrit UN caractère à l'écran (file descriptor 1 = stdout). Retourne 1 si succès, -1 si erreur.

3 — ft_print_char.c · Imprimer un caractère

■ Analogie : appuyer sur une touche du clavier

Cette fonction fait la chose la plus simple du monde : elle prend un caractère et l'affiche à l'écran. Comme appuyer sur la touche 'A' du clavier.

■ Le code

```
int ft_print_char(int c)
{
    unsigned char ch;

    ch = (unsigned char)c;
    if (write(1, &ch, 1) == -1)
        return (-1);
    return (1);
}
```

■ Explication

■ Pourquoi le paramètre est int et pas char ?

C'est une subtilité du C : va_arg promue les char en int lors du passage. La convention C dit que les caractères passés via ... arrivent comme int. C'est pour ça qu'on reçoit un int.

■ ch = (unsigned char)c;

On le reconvertit en unsigned char pour éviter les problèmes avec les caractères dont la valeur ASCII est > 127 (caractères étendus). Sans ce cast, on risquerait un comportement indéfini sur certains systèmes.

■ write(1, &ch, 1)

write prend 3 arguments : (1) où écrire (1 = écran), (2) quoi écrire (&ch; = adresse de notre octet), (3) combien d'octets (1). Si write retourne -1, quelque chose a mal tourné (disque plein, etc.).

■ return (1);

On a écrit 1 caractère, donc on retourne 1. ft_printf ajoutera ce 1 à son compteur total.

4 — ft_print_str.c · Imprimer une chaîne

■ Analogie : lire un livre lettre par lettre

Une chaîne en C c'est un tableau de caractères terminé par '\0' (le caractère nul). Cette fonction parcourt la chaîne du début jusqu'au '\0' et écrit chaque lettre.

■ Le code

```
int ft_print_str(char *s)
{
    int i;

    if (!s)
    {
        if (write(1, "(null)", 6) == -1)
            return (-1);
        return (6);
    }
    i = 0;
    while (s[i])
    {
        if (write(1, &s[i], 1) == -1)
            return (-1);
        i++;
    }
    return (i);
}
```

■ Explication

■ if (!s)

Que faire si on passe NULL comme chaîne ? En C, déréférencer NULL crashe le programme. Plutôt que de crasher, on affiche la chaîne littérale '(null)' (c'est le comportement du vrai printf de la libc). On retourne 6 car '(null)' fait 6 caractères.

■ while (s[i])

En C, une chaîne se termine par le caractère '\0' (valeur 0). La condition s[i] est vraie tant que le caractère n'est pas '\0'. C'est la façon standard de parcourir une chaîne.

■ write(1, &s[i], 1)

On passe l'adresse du i-ème caractère à write. &s[i] c'est l'adresse de la case numéro i du tableau s'.

■ return (i);

À la sortie de la boucle, i vaut exactement le nombre de caractères écrits (la longueur de la chaîne hors '\0'). On le retourne.

5 — ft_print_nbr.c · Imprimer un entier signé

■ Analogie : écrire un nombre sur une ardoise

Pour afficher le nombre -42, tu dois d'abord écrire le '-', puis écrire '42'. Mais 42 c'est aussi un problème : tu ne peux pas écrire le chiffre 4 puis le 2 directement depuis un int. Il faut les extraire un par un. ft_print_nbr gère le signe, puis délègue les chiffres à ft_print_unsigned.

■ Le code

```
int ft_print_nbr(int n)
{
    unsigned int u;
    int count;
    int sub;

    count = 0;
    if (n < 0)
    {
        if (write(1, "-", 1) == -1)
            return (-1);
        count++;
        u = (unsigned int)(-(long)n);
    }
    else
        u = (unsigned int)n;
    sub = ft_print_unsigned(u);
    if (sub < 0)
        return (-1);
    return (count + sub);
}
```

■ Explication

■ if (n < 0)

Si le nombre est négatif, on écrit d'abord le '-', on incrémente count.

■ u = (unsigned int)(-(long)n);

Piège classique du C ! Si $n = \text{INT_MIN}$ (-2147483648), faire $-n$ en int causerait un overflow (dépassement). Astuce : on cast en long d'abord (qui peut stocker $\text{INT_MIN}+1$ à INT_MAX), on le nie, puis on cast en unsigned int. Ainsi $-(-2147483648) = 2147483648$ qui tient dans un unsigned int (max ~4 milliards).

■ ft_print_unsigned(u)

Une fois le signe géré, les chiffres eux-mêmes sont toujours positifs. On délègue à ft_print_unsigned qui sait afficher des nombres sans signe.

■ **return (count + sub)**

count = 1 si on a écrit un '-', 0 sinon. sub = nombre de chiffres écrits. Le total c'est les deux additionnés.

6 — ft_print_unsigned.c · Imprimer un entier positif

■ Analogie : épeler un nombre à l'envers en récursion

Pour afficher 123, le problème c'est qu'en divisant par 10 tu obtiens les chiffres dans le mauvais ordre (3 d'abord, puis 2, puis 1). La récursion résout ça élégamment : on appelle d'abord la fonction sur 12 (ce qui va appeler sur 1), et seulement APRÈS on écrit le 3. Ainsi on écrit 1, puis 2, puis 3 dans le bon ordre.

■ Le code

```
int ft_print_unsigned(unsigned int n)
{
    char c;
    int written;
    int sub;

    written = 0;
    if (n >= 10)
    {
        sub = ft_print_unsigned(n / 10);
        if (sub < 0)
            return (-1);
        written += sub;
    }
    c = (char)(n % 10) + '0';
    if (write(1, &c, 1) == -1)
        return (-1);
    return (written + 1);
}
```

■ Explication pas à pas sur l'exemple n = 123

Voici ce qui se passe quand on appelle ft_print_unsigned(123) :

```
ft_print_unsigned(123)
  ■■■ 123 >= 10, donc appel récursif : ft_print_unsigned(12)
  ■    ■■■ 12 >= 10, donc appel récursif : ft_print_unsigned(1)
  ■    ■    ■■■ 1 < 10, pas d'appel récursif
  ■    ■    ■■■ c = (1 % 10) + '0' = 1 + 48 = '1'
  ■    ■    ■■■ write('1') → affiche '1', retourne 1
  ■    ■■■ c = (12 % 10) + '0' = 2 + 48 = '2'
  ■    ■■■ write('2') → affiche '2', retourne 1+1 = 2
  ■■■ c = (123 % 10) + '0' = 3 + 48 = '3'
  ■■■ write('3') → affiche '3', retourne 2+1 = 3
```

Résultat affiché : 123 ✓

■ **Pourquoi + '0' ?** En ASCII, le caractère '0' a la valeur 48. Le caractère '1' = 49, '2' = 50, etc. Donc pour convertir le chiffre 3 (un entier) en caractère '3', on fait 3 + 48 = 51 = '3'. C'est la conversion chiffre →

caractère affichable.

7 — ft_print_hex.c · Imprimer en hexadécimal

■ Analogie : compter avec 16 chiffres au lieu de 10

On compte normalement en base 10 (0,1,2,...,9). L'hexadécimal c'est la base 16 : 0,1,2,...,9,a,b,c,d,e,f. Après le f (=15), on repart à 0 (=16 en décimal). C'est très utilisé en informatique car 1 octet (8 bits) s'écrit toujours en 2 chiffres hex. Exemple : 255 en décimal = ff en hex.

■ Le code

```
int ft_print_hex(unsigned int n, const char format)
{
    const char *base;
    char      c;
    int        written;
    int        sub;

    if (format == 'X')
        base = "0123456789ABCDEF";
    else
        base = "0123456789abcdef";
    written = 0;
    if (n >= 16)
    {
        sub = ft_print_hex(n / 16, format);
        if (sub < 0)
            return (-1);
        written += sub;
    }
    c = base[n % 16];
    if (write(1, &c, 1) == -1)
        return (-1);
    return (written + 1);
}
```

■ Explication

■ base = '0123456789abcdef' ou '...ABCDEF'

Au lieu de faire $n \% 16 + '0'$ (qui ne fonctionnerait pas pour les lettres), on utilise une chaîne comme dictionnaire. $base[0] = '0'$, $base[10] = 'a'$, $base[15] = 'f'$. Si $format == 'X'$, on prend la version majuscules. Élégant et simple !

■ Récursion sur $n / 16$

Même principe que `ft_print_unsigned` mais en base 16. $255 / 16 = 15$ (reste 15). Appel récursif sur 15 $\rightarrow 15 < 16 \rightarrow base[15] = 'f'$. Puis retour : $base[255 \% 16] = base[15] = 'f'$. Affichage : 'ff'. ✓

■ $c = base[n \% 16]$

$n \% 16$ donne le reste (entre 0 et 15), qu'on utilise comme index dans base. C'est comme dire 'le i-ème symbole de l'alphabet hexadécimal'.

Exemple trace : `ft_print_hex(255, 'x')`

```
ft_print_hex(255, 'x')
    ■■■ base = "0123456789abcdef"
    ■■■ 255 >= 16, appel récursif : ft_print_hex(15, 'x')
    ■    ■■■ 15 < 16, pas d'appel récursif
    ■    ■■■ c = base[15 % 16] = base[15] = 'f'
    ■    ■■■ affiche 'f', retourne 1
    ■■■ c = base[255 % 16] = base[15] = 'f'
    ■■■ affiche 'f', retourne 2
```

Résultat : ff ✓ (255 en hexadécimal = ff)

8 — ft_print_ptr.c · Imprimer une adresse mémoire

■ Analogie : l'adresse d'une maison dans la RAM

La RAM de ton ordinateur est comme une très longue rue avec des millions de maisons. Chaque variable dans ton programme habite à une adresse précise. Un pointeur (ptr), c'est simplement un numéro de maison. ft_print_ptr affiche ce numéro sous la forme 0x... (le '0x' signale que c'est de l'hexa).

■ Le code

```
static int  put_hex_long(unsigned long n)
{
    const char  *base = "0123456789abcdef";
    char        c;
    int         written = 0;
    int         sub;

    if (n >= 16)
    {
        sub = put_hex_long(n / 16);
        if (sub < 0)
            return (-1);
        written += sub;
    }
    c = base[n % 16];
    if (write(1, &c, 1) == -1)
        return (-1);
    return (written + 1);
}

int  ft_print_ptr(unsigned long ptr)
{
    int  sub;

    if (ptr == 0)
    {
        if (write(1, "(nil)", 5) == -1)
            return (-1);
        return (5);
    }
    if (write(1, "0x", 2) == -1)
        return (-1);
    sub = put_hex_long(ptr);
    if (sub < 0)
        return (-1);
    return (2 + sub);
}
```

■ Explication

■ unsigned long ptr

Les adresses mémoire sont des nombres potentiellement très grands (64 bits sur les systèmes modernes). Un int (32 bits) ne suffit pas ! unsigned long est garanti d'être assez grand pour stocker une adresse.

■ if (ptr == 0) → '(nil)'

Un pointeur nul (NULL = 0) n'a pas d'adresse réelle. Par convention, on affiche '(nil)' plutôt qu'une fausse adresse. C'est le comportement du printf standard.

■ write(1, '0x', 2)

Préfixe standard pour signaler une valeur hexadécimale. On l'écrit en 2 caractères avant les chiffres hex.

■ put_hex_long vs ft_print_hex

put_hex_long est déclaré static : elle n'est visible que dans ce fichier. Elle existe parce qu'elle travaille sur unsigned long (64 bits) au lieu de unsigned int (32 bits). Les adresses mémoire dépassent parfois 32 bits, donc on a besoin du type plus grand.

■ return (2 + sub)

2 pour '0x', sub pour les chiffres hexadécimaux. Total = longueur complète affichée.

9 — ft_print_percent.c · Imprimer le symbole %

■ Analogie : un caractère d'échappement

Problème : le '%' est le signal spécial de printf. Alors comment afficher un vrai '%' ? On utilise '%%' dans le format. ft_printf détecte '%%' et appelle ft_print_percent qui affiche simplement un seul '%'.

■ Le code

```
int ft_print_percent(void)
{
    if (write(1, "%", 1) == -1)
        return (-1);
    return (1);
}
```

■ Explication

La fonction la plus simple du projet ! Elle écrit juste '%' à l'écran et retourne 1 (1 caractère écrit).
Exemple d'usage : ft_printf("100%%") affiche '100%'.

10 — Makefile · La recette de construction

■ Analogie : une recette de cuisine automatique

Le Makefile dit à l'ordinateur comment assembler tous tes fichiers .c en une bibliothèque .a. C'est comme une recette qui dit : 'D'abord fais cuire les légumes (.c → .o), puis mélange-les dans un plat (.o → .a)'.

■ Le code

```
NAME      = libftprintf.a
CC        = cc
CFLAGS    = -Wall -Wextra -Werror
AR        = ar rcs
RM        = rm -f

SRCS      = ft_printf.c ft_print_char.c ft_print_str.c \
            ft_print_nbr.c ft_print_unsigned.c ft_print_hex.c \
            ft_print_ptr.c ft_print_percent.c

OBJS      = $(SRCS:.c=.o)
HEADER    = ft_printf.h

all:      $(NAME)

$(NAME):  $(OBJS)
          $(AR) $(NAME) $(OBJS)

%.o:     %.c $(HEADER)
          $(CC) $(CFLAGS) -c $< -o $@

clean:
          $(RM) $(OBJS)

fclean:  clean
          $(RM) $(NAME)

re:      fclean all

.PHONY:  all clean fclean re
```

■ Explication des variables et règles

Variable/Règle	Valeur	Ce que ça fait
NAME	libftprintf.a	Nom de la bibliothèque finale
CC	cc	Compilateur C à utiliser
CFLAGS	-Wall -Wextra -Werror	Active tous les warnings ET les transforme en erreurs
AR	ar rcs	Crée l'archive (.a) depuis les .o

RM	rm -f	Supprime sans se plaindre si le fichier n'existe pas
SRCS	liste des .c	Tous les fichiers sources
OBJS	\$(SRCS:.c=.o)	Substitution : remplace .c par .o dans SRCS
all	→ \$(NAME)	Règle par défaut : construit la lib
\$(NAME)	→ \$(OBJS)	Regroupe les .o en .a avec ar
%.o : %.c	compilation	Compile chaque .c en .o (règle générique)
clean	rm les .o	Supprime les fichiers objets intermédiaires
fclean	clean + rm .a	Supprime tout, y compris la bibliothèque
re	fclean + all	Recompile tout depuis zéro
.PHONY	all clean...	Dit que ces règles ne sont PAS des fichiers réels

■ **Pourquoi -Wall -Wextra -Werror ?** -Wall active tous les avertissements basiques. -Wextra en rajoute d'autres. -Werror transforme les avertissements en erreurs : ton code ne compile plus tant qu'il y a le moindre warning. C'est strict mais ça t'oblige à écrire du code propre. C'est la norme à la 42.

■ Récapitulatif global

Voici comment tous les fichiers fonctionnent ensemble quand tu appelles `ft_printf("Hello %s, tu as %d ans !", "Zain", 20)` :

```
ft_printf("Hello %s, tu as %d ans !", "Zain", 20)
■
■■ Lit 'H','e','l','l','o',' ' → put_raw() pour chacun
■
■■ Lit '%s' → dispatch('s', ap)
■           → ft_print_str(va_arg(ap, char*)) = ft_print_str("Zain")
■           → écrit 'Z','a','i','n' (4 chars)
■
■■ Lit ',',' ','t','u',' ','a','s',' ' → put_raw() pour chacun
■
■■ Lit '%d' → dispatch('d', ap)
■           → ft_print_nbr(va_arg(ap, int)) = ft_print_nbr(20)
■           → ft_print_unsigned(20)
■           → récursion : ft_print_unsigned(2) écrit '2', puis écrit '0'
■           → 2 chars écrits
■
■■ Lit ' ','a','n','s','!' → put_raw() pour chacun
■
■■ Retourne 26 (nombre total de caractères écrits)
```

Bravo ! Tu as maintenant recréé une des fonctions les plus utilisées du C. Le vrai `printf` de la `libc` fait exactement la même chose, juste avec beaucoup plus de cas gérés (précision, largeur, etc.).

Fichier	Rôle	Concept clé
<code>ft_printf.h</code>	Déclarations	Guards, prototypes, includes
<code>ft_printf.c</code>	Chef d'orchestre	<code>va_list</code> , boucle, <code>dispatch</code>
<code>ft_print_char.c</code>	Imprime char	Cast unsigned char, write
<code>ft_print_str.c</code>	Imprime string	Gestion NULL, parcours jusqu'à <code>\0</code>
<code>ft_print_nbr.c</code>	Imprime int signé	Gestion signe, overflow <code>INT_MIN</code>
<code>ft_print_unsigned.c</code>	Imprime uint	Récursion, chiffre + '0'
<code>ft_print_hex.c</code>	Imprime hex	Base string, récursion base 16
<code>ft_print_ptr.c</code>	Imprime adresse	unsigned long, préfixe '0x'
<code>ft_print_percent.c</code>	Imprime %	Cas spécial %%
Makefile	Compilation	Règles make, bibliothèque .a